Azure Monitor and Log Analytics Workspace Design

Project Documentation

1. Introduction

Cloud environments consist of multiple resources such as virtual machines, databases, storage accounts, web applications, and networking services. Each resource generates logs, metrics, and operational information that can be difficult to monitor individually.

This project proposes a centralized monitoring solution using **Azure Monitor and Azure Log Analytics Workspace**. The solution collects monitoring data from approximately 40 Azure resources into a centralized workspace, where the data can be queried, analyzed, visualized, and used to generate alerts.

2. Problem Statement

Monitoring a large number of Azure resources individually can make it difficult to identify failures quickly and understand the overall health of the environment.

Traditional monitoring approaches may focus heavily on resource-utilization metrics such as CPU usage. However, high CPU usage does not necessarily indicate an application failure, while an application can fail even when CPU utilization is normal.

Therefore, the project aims to design a centralized monitoring system that:
• Collects logs from approximately 40 Azure resources.
• Stores monitoring data in a centralized Log Analytics Workspace.
• Identifies meaningful failure signals using KQL queries.
• Generates alerts based on application and operational failures.
• Provides a centralized dashboard for monitoring.
• Controls unnecessary log ingestion to reduce costs.
• Considers workspace-region selection and data-residency requirements.

3. Objectives

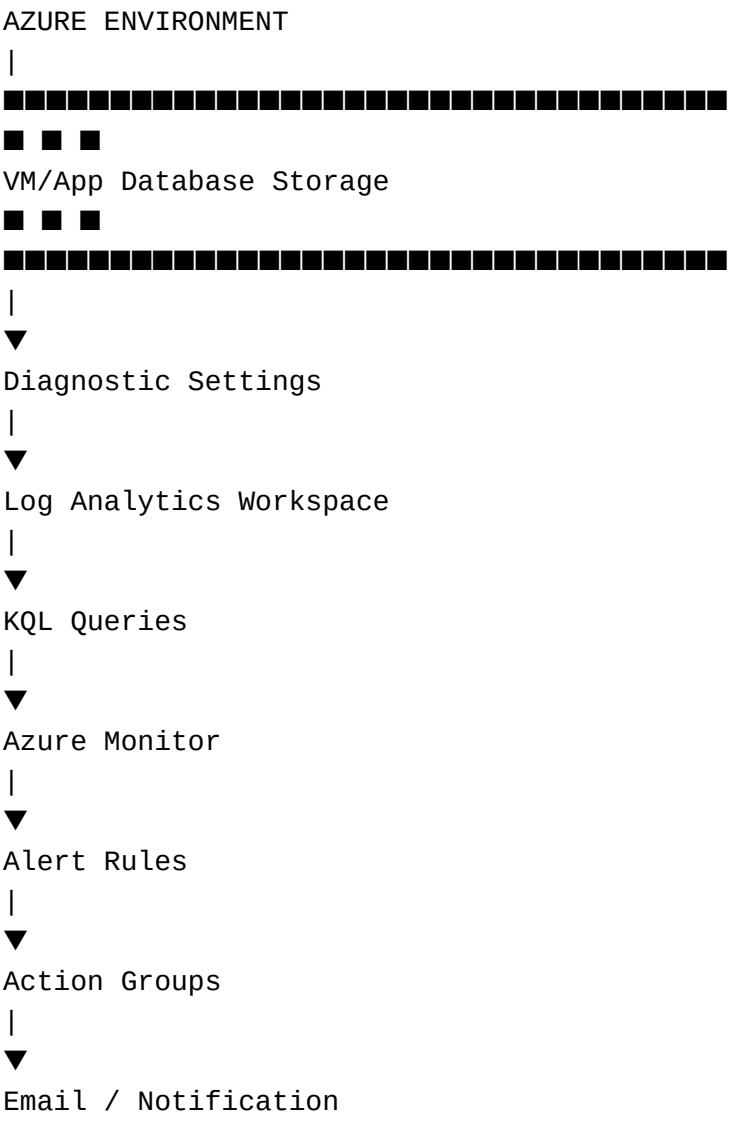
1. Create a centralized Log Analytics Workspace.
2. Connect multiple Azure resources to the workspace.
3. Collect relevant resource logs and metrics.
4. Analyze collected data using KQL.
5. Detect operational failures automatically.
6. Configure Azure Monitor alert rules.
7. Provide notifications when important failures occur.
8. Create a centralized monitoring dashboard.
9. Monitor and control log-ingestion costs.
10. Consider data residency while selecting the workspace region.

4. Proposed Solution

The proposed system uses Azure Monitor as the main monitoring service and Log Analytics Workspace as the centralized log-storage and analysis layer.

Workflow:
Azure Resources → Diagnostic Settings → Log Analytics Workspace → KQL Analysis → Azure Monitor Alerts → Notifications

5. System Architecture



6. Technologies Used

Technology	Purpose
Microsoft Azure	Cloud platform
Azure Monitor	Monitoring and alerting

Technology	Purpose
Log Analytics Workspace	Centralized log storage and analysis
Kusto Query Language (KQL)	Log analysis and failure detection
Diagnostic Settings	Collecting and routing resource logs
Azure Monitor Alerts	Detecting important conditions
Action Groups	Sending notifications
Azure Dashboard / Workbooks	Visualization

7. Implementation Process

Step 1: Create Resource Group
Create a dedicated resource group for the monitoring solution. Example: Central-Monitoring-RG.

Step 2: Create Log Analytics Workspace
Navigate to Azure Portal → Log Analytics Workspaces → Create. Configure subscription, resource group, workspace name, and region. The workspace region should be selected carefully because monitoring data may be subject to data-residency requirements.

Step 3: Connect Azure Resources
Connect approximately 40 Azure resources, such as virtual machines, App Services, storage accounts, databases, Key Vault, and networking resources, using appropriate diagnostic settings.

Step 4: Configure Diagnostic Settings
For each supported resource, select the required log and metric categories and send them to the central Log Analytics Workspace. Only relevant categories should be collected.

Step 5: Verify Data Collection
Open Log Analytics Workspace → Logs and run a basic query to confirm that monitoring data is arriving.

Step 6: Create KQL Queries
Use Kusto Query Language to identify failed operations, application errors, failed deployments, authentication failures, database errors, and other operational issues.

Step 7: Configure Failure-Based Alerts
Create Azure Monitor alerts based on meaningful failure signals rather than relying only on CPU thresholds.

Step 8: Configure Action Groups
Configure email or other supported notification actions when an alert is triggered.

Step 9: Build Monitoring Dashboard
Create a dashboard or workbook showing monitored resources, failures, critical alerts, recent errors, resource health, ingestion volume, and alert trends.

Step 10: Test the System
Generate test failures, verify that logs reach the workspace, confirm that KQL detects the failures, and verify that the configured alert and notification are triggered.

8. KQL-Based Failure Detection

A basic failed-operation query can be used to identify failed Azure operations:

```
AzureActivity
| where ActivityStatusValue == "Failed"
| project TimeGenerated,
          ResourceGroup,
          ResourceId,
          OperationNameValue,
          ActivityStatusValue
| order by TimeGenerated desc
```

9. Failure-Based Alerting

Instead of creating only CPU-based alerts such as **CPU > 80%** → **Alert**, the proposed system focuses on meaningful failure signals.

Application Failure → Log Analytics → KQL Query → Failure Detected → Azure Monitor Alert → Notification

10. Cost Optimization

A major challenge is data-ingestion cost. Collecting unnecessary logs from 40 resources can significantly increase the volume of data sent to the workspace.

The project should identify requirements, select only necessary categories, monitor ingestion, and continuously optimize collection. Log-ingestion volume should be reviewed regularly.

11. Data Residency Consideration

Workspace region selection is an important design consideration. The team should evaluate organizational location, regulatory requirements, data residency requirements, and where monitoring data should be stored. Multiple workspaces may be considered if different resources have different compliance or location requirements.

12. Testing

Test Case 1 — Successful Operation: Resource operation → Log generated → Log Analytics → No failure alert.

Test Case 2 — Failed Operation: Resource operation → Failure → Log generated → KQL detects failure → Alert triggered → Notification received.

Test Case 3 — Multiple Failures: Generate multiple failures within a defined period → threshold exceeded → Azure Monitor alert → Action Group → Notification.

13. Expected Results

• Centralized monitoring for approximately 40 resources.
• Centralized log storage.
• Faster identification of failures.
• KQL-based failure analysis.
• Automated alerts.
• Centralized monitoring dashboards.
• Better operational visibility.
• Reduced manual troubleshooting.
• Improved control over log-ingestion costs.
• Consideration of data residency requirements.

14. Advantages

Centralized Monitoring: Important monitoring information can be analyzed from one workspace.

Faster Troubleshooting: Administrators can use KQL queries to identify failures without checking every resource individually.

Proactive Alerting: Failures can trigger alerts automatically.

Scalability: The architecture can be extended as additional Azure resources are added.

Cost Awareness: Log collection can be optimized by selecting only relevant data.

Compliance Awareness: Workspace-region selection can be incorporated into the architecture according to data-residency requirements.

15. Limitations

1. Log-ingestion costs can increase as the number of resources grows.
2. Different Azure resources expose different log categories.
3. Some logs may have ingestion delays.
4. Incorrect alert thresholds can result in false positives or missed failures.
5. Workspace-region selection may restrict architectural choices because of compliance requirements.
6. Monitoring approximately 40 resources requires consistent diagnostic-setting configuration.

16. Future Enhancements

• AI-assisted anomaly detection.
• Automated incident response.
• Microsoft Sentinel integration.
• Automated remediation using Azure Logic Apps.
• Predictive failure detection.
• Advanced security monitoring.

• Multi-workspace architecture.
• Cost-optimization recommendations.
• Power BI-based monitoring reports.

17. Conclusion

This project demonstrates the design and implementation of a centralized Azure monitoring architecture using Azure Monitor and Log Analytics Workspace. By routing logs from approximately 40 Azure resources into a centralized workspace, administrators can analyze operational data using KQL and identify meaningful failure signals.

Unlike traditional monitoring approaches that rely primarily on resource-utilization thresholds, the proposed solution emphasizes failure-oriented monitoring and automated alerting. The design also addresses practical cloud-monitoring challenges such as data-ingestion costs, workspace scalability, and data-residency considerations.

The resulting architecture provides a scalable and centralized approach to Azure monitoring, enabling organizations to improve visibility, detect failures more quickly, and respond to operational issues proactively.